

06-Outlier-Detection-Lab-02

February 9, 2026

<div style='float:left;'><h1>CPSC 4300/6300: Applied Data Science</h1></div>
 upper_bound))
df_outliers_iqr = df[df['IQR_Outlier']]
print(f"Number of outliers detected by IQR: {len(df_outliers_iqr)}")
df_outliers_iqr.head()
```

### 2.1.2 Visualizing IQR Outliers

We'll mark the outliers detected by IQR in red.

```
[]: plt.figure(figsize=(8,5))
inliers = df[~df['IQR_Outlier']]
outliers = df[df['IQR_Outlier']]

plt.scatter(inliers['X'], inliers['y'], color='blue', alpha=0.7,
 label='Inliers')
plt.scatter(outliers['X'], outliers['y'], color='red', alpha=0.7,
 label='Outliers')
plt.title('IQR Outlier Detection')
plt.xlabel('X')
plt.ylabel('y')
plt.legend()
plt.show()
```

## 2.2 1.2 z-score Detection

### 2.2.1 z-score Definition

The z-score of a data point measures how many standard deviations away it is from the mean:

$$z = \frac{x - \mu}{\sigma}$$

where  $x$  is the data point,  $\mu$  is the mean, and  $\sigma$  is the standard deviation. A common rule of thumb is to consider points with  $|z| > 3$  as outliers.

z-score detection is a common technique for data scientists working with data that is (or is assumed to be) approximately normally distributed. It helps highlight points far from the mean.

```
[]: y_mean = np.mean(df['y'])
 y_std = np.std(df['y'])

 df['z_score'] = (df['y'] - y_mean) / y_std
 threshold = 3

 df['Z_Outlier'] = df['z_score'].abs() > threshold

 df_outliers_z = df[df['Z_Outlier']]
 print(f"Number of outliers detected by Z-score: {len(df_outliers_z)}")
 df_outliers_z.head()
```

### 2.2.2 Visualizing z-score Outliers

```
[]: plt.figure(figsize=(8,5))
 inliers_z = df[~df['Z_Outlier']]
 outliers_z = df[df['Z_Outlier']]

 plt.scatter(inliers_z['X'], inliers_z['y'], color='blue', alpha=0.7,
 label='Inliers')
 plt.scatter(outliers_z['X'], outliers_z['y'], color='red', alpha=0.7,
 label='Outliers')
 plt.title('z-score Outlier Detection')
 plt.xlabel('X')
 plt.ylabel('y')
 plt.legend()
 plt.show()
```

### 2.2.3 Observations

1. **IQR vs z-score:** Notice that the two methods may flag slightly different points as outliers.
2. **Domain knowledge:** In practice, combine statistical methods with domain expertise to decide if an outlier is truly erroneous or a genuine data point.

---

## 3 2. Model Selection

Once we have understood and potentially handled outliers (either by removing them or adjusting them), we often need to choose a model. We'll explore how to use `scikit-learn` pipelines, then train two simple models:

1. **Linear Regression**
2. **k-Nearest Neighbors (kNN)**

We'll then compare them using cross validation.

**Why is this important to data scientists?**

- Choosing the right model can have a huge impact on accuracy and generalization.
- Pipelines help maintain clean and reproducible workflows.
- Trying multiple models is a core practice to find the best approach for a given dataset.

### 3.1 2.1 scikit-learn Pipeline

A **Pipeline** in **scikit-learn** lets you sequence multiple steps, such as transformations (scaling, encoding, etc.) and an estimator (like a regression model), into one object. This has several advantages:

1. **Convenience and encapsulation:** All steps can be contained in one pipeline.
2. **Consistency:** Ensures the same transformations apply to both training and test sets.
3. **Optimization:** Hyperparameter tuning can be done across the entire pipeline in an automated way.

For data scientists, pipelines save time, reduce the risk of data leakage, and make code more organized.

### 3.2 2.2 Linear Regression

Linear Regression assumes:

$$y = w_0 + w_1X_1 + \dots + w_nX_n + \epsilon$$

where  $\epsilon$  is the error term. It tries to find weights  $w_i$  that minimize the residual sum of squares between the observed targets and predicted targets.

Data scientists often start with linear regression as a baseline model because it's simple, fast, and easily interpretable.

```
[]: # Split data into training and testing sets
X_data = df[['X']] # features
y_data = df['y'] # target

X_train, X_test, y_train, y_test = train_test_split(X_data, y_data,
 test_size=0.2,
 random_state=42)

Create a pipeline with a scaler and linear regression
pipeline_lr = Pipeline([
 ('scaler', StandardScaler()),
 ('lr', LinearRegression())
])
```

```

Fit the model
pipeline_lr.fit(X_train, y_train)

Predict on test set
y_pred_lr = pipeline_lr.predict(X_test)

Evaluate performance (e.g., using mean squared error)
mse_lr = np.mean((y_test - y_pred_lr)**2)
print(f"Linear Regression MSE: {mse_lr:.2f}")

```

### 3.3 2.3 k-Nearest Neighbors (kNN)

kNN is a non-parametric method that predicts the target by looking at the  $k$  nearest training examples in the feature space. For regression, it takes the average of these neighbors as the prediction.

This model is often useful for data scientists who want a simple, instance-based approach that can adapt to complex data distributions without a strict functional form.

```

[]: # Question: Is k=5 the appropriate number of neighbors?
We'll start by using k=5 as a default and then test a few different k values.

Create a pipeline with a scaler and kNN (for regression)
pipeline_knn_5 = Pipeline([
 ('scaler', StandardScaler()),
 ('knn', KNeighborsRegressor(n_neighbors=5)) # default k=5
])

Fit the model
pipeline_knn_5.fit(X_train, y_train)

Predict on test set
y_pred_knn_5 = pipeline_knn_5.predict(X_test)

Evaluate performance
mse_knn_5 = np.mean((y_test - y_pred_knn_5)**2)
print(f"kNN Regression (k=5) MSE: {mse_knn_5:.2f}")

```

#### 3.3.1 Testing Different Values of $k$

Now, let's see how changing the number of neighbors ( $k$ ) affects the performance of kNN. We'll train multiple kNN regressors with different  $k$  values and compare their mean squared errors on the test set.

```

[]: k_values_to_test = list(range(1, 400, 10))
mse_scores = []

for k_val in k_values_to_test:
 pipeline_knn_var = Pipeline([

```

```

 ('scaler', StandardScaler()),
 ('knn', KNeighborsRegressor(n_neighbors=k_val))
])
 # Fit the model
 pipeline_knn_var.fit(X_train, y_train)
 # Predict
 y_pred = pipeline_knn_var.predict(X_test)
 # Calculate MSE
 mse = np.mean((y_test - y_pred)**2)
 mse_scores.append(mse)

Let's visualize the impact of adjusting k on the error
plt.figure(figsize=(8,5))
plt.plot(k_values_to_test, mse_scores, marker='o', linestyle='--', color='blue')
plt.title('Impact of k on kNN MSE')
plt.xlabel('Number of Neighbors (k)')
plt.ylabel('Mean Squared Error')
plt.show()

```

### 3.3.2 Observations about $k$

- Smaller  $k$  values (e.g.,  $k = 1$  or  $k = 3$ ) often capture more local patterns but can be noisy.
- Larger  $k$  values (e.g.,  $k = 15$ ) provide smoother predictions but might underfit.
- Choosing an optimal  $k$  usually involves balancing variance (too small  $k$ ) and bias (too large  $k$ ), and often requires empirical testing or cross validation.

So, is  $k = 5$  the best choice? It depends on the data. In this example, you can see from the plot which  $k$  gives the lowest MSE on the test set. In practice, you might also use cross validation to find the best  $k$  in a more robust manner.

### 3.3.3 Observations

- We have two models with potentially different performance.
- We evaluated them on a single train-test split.
- To get a more reliable estimate of model performance, we use **cross validation**.

## 4 3. Cross Validation

Cross validation is a technique to evaluate models by splitting the data multiple times and aggregating results. This helps provide a more robust measure of how well the model generalizes to unseen data.

In practice, data scientists rely on cross validation to:

- Get a more stable performance metric.
- Make better model selection decisions.
- Mitigate overfitting by not relying on a single train-test split.

Common methods:



1. k-fold Cross Validation
2. Leave-One-Out (LOO) Cross Validation

### 4.1 3.1 k-fold Validation

The dataset is split into  $k$  folds (subsets). The model is trained on  $k - 1$  folds and validated on the remaining fold. This process is repeated  $k$  times, with each fold used exactly once as the validation set.

Let's first use 5-fold cross validation for both our Linear Regression and kNN models and compare their performance.

This approach provides a balance between computational cost and robust performance estimation, making it a favorite for many data scientists.

```
[]: k = 5
kf = KFold(n_splits=k, shuffle=True, random_state=42)

pipeline_lr = Pipeline([
 ('scaler', StandardScaler()),
 ('lr', LinearRegression())
])

pipeline_knn_cv = Pipeline([
 ('scaler', StandardScaler()),
 ('knn', KNeighborsRegressor(n_neighbors=5))
])

Cross validation scores
scores_lr = cross_val_score(pipeline_lr, X_data, y_data, cv=kf,
 ↪scoring='neg_mean_squared_error')
scores_knn = cross_val_score(pipeline_knn_cv, X_data, y_data, cv=kf,
 ↪scoring='neg_mean_squared_error')

Convert scores from negative MSE to positive MSE
mse_lr_cv = -scores_lr
mse_knn_cv = -scores_knn

print(f"Linear Regression Mean MSE (5-fold): {np.mean(mse_lr_cv):.2f} ± {np.
 ↪std(mse_lr_cv):.2f}")
print(f"kNN Regression Mean MSE (5-fold): {np.mean(mse_knn_cv):.2f} ± {np.
 ↪std(mse_knn_cv):.2f}")
```

#### 4.1.1 Visualizing 5-fold Validation Results

We can visualize the MSE for each fold to get a sense of the model performance distribution.

```
[]: # Create a dataframe for visualization
cv_results = pd.DataFrame({
```

```

 'Fold': list(range(1, k+1)) + list(range(1, k+1)),
 'MSE': np.concatenate([mse_lr_cv, mse_knn_cv]),
 'Model': ['LinearReg'] * k + ['kNN'] * k
})

plt.figure(figsize=(8,5))
sns.barplot(x='Fold', y='MSE', hue='Model', data=cv_results)
plt.title('MSE for Each Fold by Model (5-Fold)')
plt.show()

```

#### 4.1.2 Demonstrating Different $k$ Values for Cross Validation

While 5-fold cross validation is quite common, the choice of  $k$  can affect:

1. **Bias-Variance trade-off** in the performance estimate.
2. **Computational cost** (larger  $k$  can be more expensive).
3. **Stability** of the results.

Let's see how different values of  $k$  (from 2 to 19) affect the cross validation results for both Linear Regression and kNN (with  $k = 5$  neighbors). We'll visualize and compare the performance across these different  $k$  values.

```

[]: k_values = list(range(2, 20))

results_dict = {'k': [], 'Model': [], 'Mean MSE': []}

for k_val in k_values:
 kf_var = KFold(n_splits=k_val, shuffle=True, random_state=42)

 # Linear Regression
 scores_lr_var = cross_val_score(pipeline_lr, X_data, y_data, cv=kf_var,
 ↪scoring='neg_mean_squared_error')
 mse_lr_var = -scores_lr_var
 mean_mse_lr = np.mean(mse_lr_var)
 results_dict['k'].append(k_val)
 results_dict['Model'].append('Linear Regression')
 results_dict['Mean MSE'].append(mean_mse_lr)

 # kNN with k=5 neighbors
 scores_knn_var = cross_val_score(pipeline_knn_cv, X_data, y_data,
 ↪cv=kf_var, scoring='neg_mean_squared_error')
 mse_knn_var = -scores_knn_var
 mean_mse_knn = np.mean(mse_knn_var)
 results_dict['k'].append(k_val)
 results_dict['Model'].append('kNN')
 results_dict['Mean MSE'].append(mean_mse_knn)

kfold_comparison_df = pd.DataFrame(results_dict)

```

```
kfold_comparison_df.head()
```

### 4.1.3 Visualizing the Effect of Different $k$ Values (CV)

Let's create a bar plot to compare the mean MSE for each  $k$  (folds) and each model.

```
[]: plt.figure(figsize=(8,5))
sns.barplot(x='k', y='Mean MSE', hue='Model', data=kfold_comparison_df)
plt.title('Comparison of Mean MSE for Different Cross-Validation k Values')
plt.xlabel('k (number of folds)')
plt.ylabel('Mean MSE')
plt.legend(loc='lower center')
plt.show()
```

### 4.1.4 Observations on Different $k$ Values

- For large  $k$  (close to the dataset size), you're nearing Leave-One-Out Cross Validation.
- For smaller  $k$ , the training set in each fold is larger, and the validation set is smaller.
- Typically, 5-fold or 10-fold are common defaults in many scenarios.

### 4.1.5 Quick Question

Now that we've seen the error's that these two models are working with on the data - which is the more appropriate choice for the data?

1. KNN
2. Linear Regression

Put your answer in a variable called `answer`

```
[]: # your code here
raise NotImplementedError
```

```
[]:
```

## 4.2 3.2 Leave-One-Out Validation

Leave-One-Out Cross Validation (LOOCV) is a special case of  $k$ -fold cross validation where  $k$  is equal to the number of data points. Each time, we train on all the points except one, and use that one point for validation. This can be computationally expensive but provides an almost unbiased estimate of model performance.

For data scientists, LOOCV can be appealing for smaller datasets where maximizing training data usage is critical, but it becomes expensive for larger datasets.

```
[]: loo = LeaveOneOut()
pipeline_lr_loo = Pipeline([
 ('scaler', StandardScaler()),
 ('lr', LinearRegression())
])
```

```

])

pipeline_knn_loo = Pipeline([
 ('scaler', StandardScaler()),
 ('knn', KNeighborsRegressor(n_neighbors=5))
])

scores_lr_loo = cross_val_score(pipeline_lr_loo, X_data, y_data, cv=loo,
 ↪scoring='neg_mean_squared_error')
scores_knn_loo = cross_val_score(pipeline_knn_loo, X_data, y_data, cv=loo,
 ↪scoring='neg_mean_squared_error')

mse_lr_loo = -scores_lr_loo
mse_knn_loo = -scores_knn_loo

print(f"Linear Regression Mean MSE (LOOCV): {np.mean(mse_lr_loo):.2f}")
print(f"kNN Regression Mean MSE (LOOCV): {np.mean(mse_knn_loo):.2f}")

```

### 4.3 Summary (of the demos)

We've covered a lot of ground in this notebook:

1. Detecting outliers using **IQR** and **z-score** methods.
2. Setting up and using a **scikit-learn Pipeline** for data scaling and model fitting.
3. Training and comparing **Linear Regression** and **k-Nearest Neighbors**.
4. Evaluating model performance using **k-fold** (with different  $k$  values) and **Leave-One-Out Cross Validation**.
5. Seeing if different values of  $k$  are appropriate choices for kNN by testing different neighbor counts.

## 5 Exercises

Below are some exercises for you to practice the techniques covered in this notebook.

### 5.1 Exercise 1: Outlier Handling and Impact on Model Performance

#### 5.1.1 Instructions

In this exercise, you will investigate how removing outliers affects model performance by following these steps:

1. **Choose an outlier detection method:**
  - Use either the **Interquartile Range (IQR) method** or **z-score method** to detect and remove outliers from the dataset.
  - Store the cleaned dataset as a new DataFrame, **df\_clean**, to allow for easy comparison with the original dataset.
2. **Split the cleaned dataset:**
  - Separate **X\_clean** (features) and **y\_clean** (target variable).

- Perform an 80-20 train-test split on the cleaned dataset using `train_test_split()` with `random_state=42`.
3. **Train and evaluate models on the cleaned dataset:**
    - Train **Linear Regression** on the cleaned data using a scikit-learn **Pipeline**.
    - Train **k-Nearest Neighbors (kNN)** with `n_neighbors=5` on the cleaned data.
    - Compute the **Mean Squared Error (MSE)** for both models and store them in variables:
      - `mse_lr_clean` → MSE for Linear Regression on cleaned data.
      - `mse_knn_clean` → MSE for kNN on cleaned data.
  4. **Compare results:**
    - Print the dataset sizes before and after outlier removal.
    - Print and compare MSE values for both models on **cleaned vs. original data**.

### 5.1.2 Key Considerations

- Ensure `df_clean` has fewer rows than `df` (i.e., outliers are actually removed).
- Expect **MSE to decrease** after removing outliers.
- Store the computed MSE values in `mse_lr_clean` and `mse_knn_clean` to pass the test cases.
- The test cases will check:
  - If `df_clean` exists and is smaller than `df`.
  - If `mse_lr_clean` and `mse_knn_clean` are correctly computed and stored.
  - If MSE values on cleaned data are lower than on the original data.

### 5.1.3 Hints

- You can use **IQR detection**: “python `Q1 = df['y'].quantile(0.25)` `Q3 = df['y'].quantile(0.75)` `IQR = Q3 - Q1` `lower_bound = Q1 - 1.5 * IQR` `upper_bound = Q3 + 1.5 * IQR` `df_clean = df[(df['y'] >= lower_bound) & (df['y'] <= upper_bound)].copy()`”

```
[]: # your code here
raise NotImplementedError
```

```
[]: # === EXERCISE 1 DEEPER CHECKS (HIDDEN) ===
```

```
[]: # === EXERCISE 1 SHALLOW CHECKS ===
assert 'df_clean' in globals(), "df_clean is not defined. Make sure you created_
↳ a cleaned DataFrame."
assert df_clean.shape[0] <= df.shape[0], "Cleaned dataset should not be larger_
↳ than the original."
```

Which of the following are potential reasons removing outliers might improve model performance?

1. It reduces the variance caused by extreme data points.
2. It can simplify the learning task for models sensitive to outliers.
3. It always increases the accuracy of every possible model.
4. It can help certain models meet their statistical assumptions.
5. It guarantees we will never overfit.

Put your choices into an array of integers called `answer`

```
[]: # Provide the correct choices as a list of integers. For instance, answer = [1, 2, 4]
 ↪2, 4]
 # your code here
 raise NotImplementedError
```

```
[]:
```

## 5.2 Exercise 2: Trying a New Transformer or Model in the Pipeline

### 5.2.1 Instructions

In this exercise, you will experiment with adding **polynomial features** to the pipeline and evaluate whether they improve model performance.

Follow these steps:

1. **Create a new pipeline:**
  - Add **PolynomialFeatures(degree=2)** from `sklearn.preprocessing` as a transformation step.
  - Normalize the data using `StandardScaler()`.
  - Train a **Linear Regression** model using the transformed features.
2. **Train and evaluate the model:**
  - Use the original training dataset (`X_train`, `y_train`).
  - Fit the polynomial regression pipeline.
  - Predict on the test dataset (`X_test`).
  - Compute the **Mean Squared Error (MSE)** of the polynomial regression model.
  - Store the result in the variable `mse_poly`.
3. **Compare results:**
  - Print the **MSE of the polynomial regression model**.
  - Compare it with the **MSE of the original Linear Regression (`mse_lr`)**.
  - Analyze whether polynomial regression improves or worsens performance.

### 5.2.2 Key Considerations

- Ensure `pipeline_poly_lr` exists and includes `PolynomialFeatures(degree=2)`, `StandardScaler()`, and `LinearRegression()`.
- Store the computed **MSE in `mse_poly`** to pass the test cases.
- The test cases will check:
  - If `pipeline_poly_lr` is defined correctly.
  - If `mse_poly` is computed and stored.
  - If `mse_poly` is **not exactly equal to `mse_lr`**, ensuring that polynomial features were used.

### 5.2.3 Questions for Analysis

- Does polynomial regression improve or worsen the performance on this dataset?
- Why might polynomial features be helpful or harmful?
  - Polynomial features can capture **non-linear relationships** in data, potentially reducing bias.

- However, they can also lead to **overfitting**, increasing variance and reducing generalizability.
- Watch for overfitting if the **polynomial degree is too high**.

#### 5.2.4 Hints

- Your pipeline should look like this: “python pipeline\_poly\_lr = Pipeline([ ('poly', PolynomialFeatures(degree=2)), ('scaler', StandardScaler()), ('lr', LinearRegression())])

```
[]: # your code here
raise NotImplementedError
```

```
[]: # === EXERCISE 2 DEEPER CHECKS (HIDDEN) ===
```

```
[]: # === EXERCISE 2 SHALLOW CHECKS ===
assert 'pipeline_poly_lr' in globals(), "pipeline_poly_lr is not defined. Make_
 ↪sure to create a pipeline with PolynomialFeatures."
assert 'mse_poly' in globals(), "mse_poly is not defined. Make sure you stored_
 ↪the polynomial regression MSE."
```

Which statements about adding polynomial features to a linear regression are correct?

1. It can model non-linear relationships.
2. It never changes training time.
3. It can lead to overfitting if the polynomial degree is too high.
4. It's guaranteed to reduce MSE on every dataset.
5. It can increase model complexity.

Put your choices into an array of integers called **answer**

```
[]: # your code here
raise NotImplementedError
```

```
[]:
```

### 5.3 Exercise 3: Exploring Different Metrics in Cross Validation

#### 5.3.1 Instructions

In this exercise, you will evaluate models using different performance metrics in **k-fold cross-validation (5-fold)** and analyze how the choice of metric affects model ranking.

Follow these steps:

1. **Perform 5-fold cross-validation** on both models (**Linear Regression** and **kNN**) using:
  - **Mean Absolute Error (MAE)** as the scoring metric (scoring='neg\_mean\_absolute\_error').
  - **R<sup>2</sup> Score** as the scoring metric (scoring='r2').
  - Store the results for each fold.
2. **Compute and store the results:**
  - Convert the **negative MAE scores** to positive values.

- Compute the **mean and standard deviation** for both MAE and  $R^2$  metrics.
  - Print the results to compare performance.
3. **Compare results:**
- Analyze if **Linear Regression or kNN performs better under different metrics**.
  - Observe if the “best” model changes based on **MAE vs.  $R^2$  score**.

### 5.3.2 Key Considerations

- Ensure **5-fold cross-validation** is used.
- Store the computed **MAE and  $R^2$  values** correctly:
  - `scores_lr_mae` and `scores_knn_mae` should contain **negative values** (due to `cross_val_score`).
  - Convert these to positive values before comparing.
  - `scores_lr_r2` and `scores_knn_r2` should contain  **$R^2$  values**.
- The test cases will check:
  - If **5-fold cross-validation** is applied correctly.
  - If the correct metrics (MAE and  $R^2$ ) are used.
  - If MAE values are **negative** before conversion.

### 5.3.3 Questions for Analysis

- Which metric shows the greatest difference between the two models?
- Why might one model outperform the other on a specific metric?
  - MAE focuses on **absolute errors**, penalizing large errors equally.
  - $R^2$  measures how well the model explains variance in the data.
  - kNN may perform better in cases with **non-linear patterns**, while Linear Regression might do better if the data is **linearly structured**.

### 5.3.4 Hints

- Use the following scoring methods in `cross_val_score()`: “python  
`cross_val_score(pipeline_lr, X_data, y_data, cv=5, scoring='neg_mean_absolute_error')`  
`cross_val_score(pipeline_lr, X_data, y_data, cv=5, scoring='r2')`”

```
[]: # your code here
raise NotImplementedError
```

```
[]: # === EXERCISE 3 DEEPER CHECKS (HIDDEN) ===
```

```
[]: # === EXERCISE 3 SHALLOW CHECKS ===
We'll just check if the cross-validation arrays exist.
assert 'scores_lr_mae' in globals(), "scores_lr_mae not found. Make sure you're_
↳doing cross_val_score with MAE for LR."
assert 'scores_knn_mae' in globals(), "scores_knn_mae not found. Make sure_
↳you're doing cross_val_score with MAE for kNN."
```

Which of the following metrics can be used to evaluate regression models?

1. Mean Squared Error (MSE)



2. Accuracy
3. Mean Absolute Error (MAE)
4.  $R^2$  Score
5. F1-Score

Put your choices into an array of integers called `answer`

```
[]: # Provide the correct choices as a list of integers. For instance, [1, 3, 4].
your code here
raise NotImplementedError
```

```
[]:
```

## 5.4 Exercise 4: Manual Hyperparameter Search for $k$ in kNN Using Cross Validation

### 5.4.1 Instructions

In this exercise, you will **experiment with different values of  $k$**  in k-Nearest Neighbors (kNN) and determine the best choice using **5-fold cross-validation**.

Follow these steps:

1. **Test different values of  $k$ :**
  - Use `k_values_search = range(1, 31)`, testing values of  $k$  from 1 to 30.
  - For each  $k$ , train a kNN model and evaluate its **Mean Squared Error (MSE)** using **5-fold cross-validation**.
  - Store the **average MSE** for each  $k$  in a list called `mse_means`.
2. **Plot the results:**
  - Create a **line plot** of  $k$  (number of neighbors) vs. MSE.
  - Label the x-axis as “ $k$  (number of neighbors)” and the y-axis as “Mean MSE”.
  - Identify the  $k$  value that yields the lowest MSE.
3. **Determine the best  $k$ :**
  - Find and print the **optimal  $k$**  (the one with the lowest MSE).
  - Print the **minimum MSE** observed.

### 5.4.2 Key Considerations

- Ensure **5-fold cross-validation** is used to compute MSE.
- Store the computed values in:
  - `k_values_search` → The range of  $k$  values tested.
  - `mse_means` → The list of corresponding **MSE values**.
  - `best_k` → The optimal  $k$  value (minimizing MSE).
- The test cases will check:
  - If `k_values_search` contains **values from 1 to 30**.
  - If `mse_means` has **the same length** as `k_values_search`.
  - If the **minimum MSE** is **non-negative**.

### 5.4.3 Questions for Analysis

- Which **k** provides the best balance of bias vs. variance?
- Does this optimal **k** differ from what you found using a single train-test split?
  - Small **k** (e.g., 1-5) captures **local patterns** but may lead to **high variance**.
  - Large **k** (e.g., 20-30) smooths predictions but may lead to **high bias**.

```
[]: # your code here
 raise NotImplementedError
```

```
[]: # === EXERCISE 4 DEEPER CHECKS (HIDDEN) ===
```

```
[]: # === EXERCISE 4 SHALLOW CHECKS ===
 # We'll confirm the array of k_values_search and mse_means exist.
 assert 'k_values_search' in globals(), "k_values_search not found."
 assert 'mse_means' in globals(), "mse_means not found."
```

### 5.4.4 EXERCISE 4 SELECT ALL THAT APPLY

Which statements about **k** in **kNN** regression are correct?

1. Smaller **k** often reduces bias but increases variance.
2. Larger **k** always yields perfect predictions.
3. **k**=1 typically yields a very high-variance model.
4. Choosing **k** with cross validation can help avoid overfitting.
5. **k** must always be an even number.

Put your choices into an array of integers called **answer**

```
[]: # your code here
 raise NotImplementedError
```

```
[]:
```

## 5.5 Exercise 5: Using GridSearchCV for Hyperparameter Tuning

### 5.5.1 Instructions

In this exercise, you will use **GridSearchCV** to systematically search for the best **k** in **k-Nearest Neighbors (kNN)** by evaluating different values of **k** using cross-validation.

Follow these steps:

1. **Define a parameter grid:**
  - Create a dictionary **param\_grid** that specifies different **k values** to test:  
[ **k** = [1, 3, 5, 10, 15, 20, 25, 30] ]
  - The key should be **\*\*'knn\_\_n\_neighbors'**, which refers to the **kNN** hyperparameter.
2. **Set up the GridSearchCV pipeline:**
  - Create a **Pipeline** with:
    - **StandardScaler()** for scaling.
    - **KNeighborsRegressor()** for **kNN** regression.

- Use `GridSearchCV()` to perform **5-fold cross-validation**:
  - Set `scoring='neg_mean_squared_error'` to evaluate performance.
  - Use `cv=5` for 5-fold cross-validation.
  - Set `n_jobs=-1` to utilize all CPU cores for faster computation.
- 3. **Fit the model and extract results**:
  - Train the `GridSearchCV` model on `X_data` and `y_data`.
  - Retrieve and print:
    - **Best k** (`grid_search.best_params_`).
    - **Best negative MSE score** (`grid_search.best_score_`).
    - **Best MSE (converted from negative score)**.

### 5.5.2 Key Considerations

- Ensure the **parameter grid** includes the correct `k` values.
- Store the trained `GridSearchCV` model in **`grid_search`**.
- The test cases will check:
  - If `grid_search.best_params_` is **not** `None`.
  - If `grid_search.best_params_` contains the `***knn__n_neighbors***` key.
  - If the best **k value** is within the tested range.

### 5.5.3 Questions for Analysis

- Does the `GridSearchCV` approach find the same best `k` as in Exercise 4?
- How does cross-validation in `GridSearchCV` help prevent overfitting?
  - Prevents bias from a single train-test split.
  - Ensures the model generalizes better by averaging results across multiple splits.

### 5.5.4 Hints

- Define the parameter grid: `python param_grid = { 'knn__n_neighbors': [1, 3, 5, 10, 15, 20, 25, 30] }`

```
[]: # your code here
raise NotImplementedError
```

```
[]: # === EXERCISE 5 DEEPER CHECKS (HIDDEN) ===
```

```
[]: # === EXERCISE 5 SHALLOW CHECKS ===
assert 'grid_search' in globals(), "grid_search not found. Make sure you_
↳ created a GridSearchCV object."
assert hasattr(grid_search, 'best_params_'), "grid_search should have_
↳ best_params_ after fitting."
```

Which statements about `GridSearchCV` are correct?

1. It uses cross-validation to evaluate different hyperparameter combinations.
2. It automatically cleans the data for you.
3. It helps find the best hyperparameters by trying each combination.

4. It guarantees your model will never overfit.
5. It can use multiple scoring metrics simultaneously if configured.

Put your choices into an array of integers called `answer`

```
[]: # your code here
 raise NotImplementedError
```

```
[]:
```

## 5.6 Exercise 6: See how much error is gained by removing outliers

### 5.6.1 Instructions

In this exercise, you will see how removing outliers is a way to reduce error. Keep in mind that when working with real data, you should know why outliers are being removed (faulty sensor, mis-entered data, etc.).

Follow these steps:

1. **Remove outliers from the data before the pipeline:**
  - Use a typical function to remove data points outside of the IQR
2. **Set up a pipeline with automatic outlier removal:**
  - Chain the following steps in a Pipeline:
    - `StandardScaler()` → Scales the data.
    - `LinearRegression()` → Fits a regression model.
3. **Compare performance:**
  - Train and evaluate this pipeline using **train-test split**.
  - Compute **Mean Squared Error (MSE)** for this pipeline.
  - Compare it to the MSE of **Linear Regression without outlier removal** (`mse_lr`).

### 5.6.2 Questions for Analysis

- How does removing outliers inside the pipeline affect your final MSE?
- Is the effect as large as you thought it would be?

```
[]: # your code here
 raise NotImplementedError
```

```
[]: # === EXERCISE 6 DEEPER CHECKS (HIDDEN) ===
```

Which statements about removing outliers in a pipeline are correct?

1. It can help avoid data leakage by only removing outliers from the training set.
2. It is impossible to remove outliers within a scikit-learn pipeline.
3. You need a custom transformer or a function transformer to do it properly.
4. Removing outliers in the pipeline is always guaranteed to improve MSE.
5. The approach may differ from removing outliers prior to splitting the data.

Put your choices into an array of integers called `answer`

```
[]: # Provide the correct choices as a list of integers. For example, [1, 3, 5].
 # your code here
 raise NotImplementedError
```

```
[]:
```

**6 END**